Writing OpenCode Agent Skills: A Practical Guide with Examples

Everything you need on your toolbelt to create reusable AI capabilities

JP Caparas

Everything you need on your toolbelt to create reusable AI capabilities

Skills are now everywhere, and they benefit everyone.

If you've spent more than a few sessions with OpenCode or any other agentic coding harness for that matter, you've

probably noticed a pattern: you keep explaining the same things. “Our team uses conventional commits.” “Always filter out test accounts from analytics queries.” “The API response comes back nested three levels deep, and you need to unwrap it like this…”

(Sigh) It gets old.

Agent Skills solve this. They’re files that capture your expertise *once*, so you **never have to repeat yourself**. Think of them as **onboarding documents** for your AI assistant, except the AI actually reads them and follows them.

This guide walks you through writing skills for OpenCode. But here’s the thing: OpenCode implements the Agent Skills open standard, which means **the same skills work across Claude Code, VS Code, Cursor, Gemini CLI, and about a dozen other tools**. Learn this once, use it everywhere.

*(**Note**: Not all models tend to leverage OpenCode’s agent skills harness correctly. I’ve noticed that Codex and Opus/Haiku/Sonnet outdo other models in progressive disclosure, and yes, that’s me dissing Gemini models).*

What Are Agent Skills, Really?

At their core, **skills are just folders containing a `SKILL.md`**

file. That's it. No special tooling, no compilation, no deployment pipeline. Create a folder, add a markdown file with some metadata at the top, and your AI assistant gains new capabilities.

The best analogy I've heard is the onboarding guide. When you hire someone new, you don't explain your company's entire tech stack, coding conventions, and deployment process in every conversation. You write it down once. Skills work the same way.

Here's what makes skills different from regular system prompts or project instructions:

Aspect	System Prompts / CLAUDE.md	Agent Skills
Loading	Always loaded at startup	Loaded on-demand when relevant
Scope	Project-wide context	Task-specific capabilities
Token cost	Consumes context constantly	Only uses tokens when activated
Reusability	Tied to one project	Portable across projects and tools
Composability	Single file	Multiple skills can combine

The on-demand loading is the key trait. Your AI doesn't need to know how to process PDFs *until* you hand it a PDF. Skills let the AI discover what's available, then load the full instructions only when needed.

The Anatomy of a SKILL.md File

Let's start with the simplest possible skill:

```
---
name: conventional-commits
description: Generate commit messages following conv
---

# Conventional Commits

Format: `type(scope): description`

Types:
- feat: new feature
- fix: bug fix
- docs: documentation
- refactor: code restructuring
- test: adding tests
- chore: maintenance

Example: `feat(auth): add OAuth2 login flow`
```

That's a working skill. The `---` delimiters mark YAML frontmatter, which contains metadata. Everything after the closing `---` is the actual content the AI loads when it activates the skill.

Directory Structure

Skills live in folders named after the skill:

```
conventional-commits/
└── SKILL.md
```

OpenCode searches these locations (in order):

1. **Project-level**: `.opencode/skill/<name>/SKILL.md`
2. **Project-level (Claude-compatible)**: `.claude/skills/<name>/SKILL.md`
3. **Global**: `~/.config/opencode/skill/<name>/SKILL.md`
4. **Global (Claude-compatible)**: `~/.claude/skills/<name>/SKILL.md`

The Claude-compatible paths mean you can share skills between OpenCode and Claude Code without copying files.

Required Frontmatter Fields

Every skill needs exactly two fields:

`name` (required)

- 1–64 characters
- Lowercase alphanumeric with hyphens only
- Can't start or end with a hyphen
- Can't contain consecutive hyphens (`--`)
- Must match the folder name

The regex if you want it: `^[a-z0-9]+(-[a-z0-9]+)*$`

`description` (required)

- 1–1024 characters
- Should describe what the skill does AND when to use it

The description is the first thing the AI sees, and it determines whether your skill gets activated. Write it carefully.

Optional Frontmatter Fields

```
---
name: pdf-processing
description: Extract text and tables from PDFs, fil
license: MIT
compatibility: Requires poppler-utils for command-l
metadata:
  author: your-name
  version: "1.0"
  category: document-processing
---
```

Most skills don’t need these. Add them when they’re genuinely useful.

Progressive Disclosure: Why Skills Scale

Here’s where it gets interesting. Skills use a three-level architecture that keeps token usage efficient:

How it all flows downstream

Level 1: Metadata At startup, the AI loads just the `name` and `description` from every installed skill. This costs roughly 100 tokens per skill. With 50 skills installed, that's 5,000 tokens, which leaves plenty of room.

Level 2: Instructions When the AI decides a skill is relevant to the current task, it reads the full `SKILL.md` body. Anthropic recommends keeping this under 5,000 tokens (roughly 500 lines).

Level 3: Resources Skills can bundle additional files: scripts, reference documentation, templates, assets. The AI loads these only when the instructions explicitly reference them.

This layered approach means you can bundle a mountain of documentation without paying the token cost upfront. A skill with 50,000 tokens of reference material costs nothing until

someone actually needs that reference.

Example Gallery: Eight Skills, Eight Configurations

Here are eight different ways to structure skills, from dead simple to full-featured.

Example 1: Minimal Skill (Just the Essentials)

```
---
name: nz-english
description: Write using New Zealand English spellir
---

# NZ English Conventions

- **-ise** not -ize (organise, recognise, realise)
- **-our** not -or (colour, favour, behaviour)
- **-re** not -er (centre, metre, theatre)
- **-l-** doubled before suffixes (travelling, cance
- "focused" not "focussed"
- Date format: DD/MM/YYYY or "15 January 2026"
```

Forty lines, does one thing well.

Example 2: Full Frontmatter

```
---
name: api-documentation
description: Generate OpenAPI-compliant API document
license: Apache-2.0
compatibility: Works with TypeScript, Python, and Go
metadata:
  author: platform-team
  version: "2.1.0"
  last-updated: "2025-12-01"
  category: documentation
---

# API Documentation Generator

## When to Use This Skill

Activate when:
- Creating new API endpoints
- Updating existing endpoint documentation
- Generating OpenAPI/Swagger specs

## Documentation Template

For each endpoint, include:
1. HTTP method and path
2. Description of what it does
3. Request parameters (path, query, body)
4. Response schema with examples
5. Error responses
6. Authentication requirements
```

```
[Rest of instructions...]
```

The `metadata` field is useful for tracking versions and ownership when you have multiple contributors.

Example 3: With Bundled Scripts

````
---
name: database-migrations
description: Create and run database migrations safe
license: MIT
compatibility: Requires Node.js 18+ and running Post
---

# Database Migrations

## Quick Start

Generate a new migration:
```bash
node scripts/create-migration.js "add_user_preferenc
```

## Safety Checklist

Before running any migration:
1. Run `node scripts/validate-migration.js` to check
2. Take a backup: `node scripts/backup-db.js`
3. Run migration: `node scripts/run-migration.js`
````

```
4. Verify: `node scripts/verify-schema.js`

## Scripts Reference

| Script | Purpose |
|--------|---------|
| `create-migration.js` | Scaffolds new migration fi
| `validate-migration.js` | Checks for common mistak
| `backup-db.js` | Creates point-in-time backup |
| `run-migration.js` | Applies pending migrations |
| `verify-schema.js` | Confirms schema matches expec
```

The folder structure:

```
database-migrations/
├── SKILL.md
└── scripts/
    ├── create-migration.js
    ├── validate-migration.js
    ├── backup-db.js
    ├── run-migration.js
    └── verify-schema.js
```

The AI can execute these scripts without loading their code into context. Only the output uses tokens.

Example 4: With Reference Files (Domain-Specific)

````
---
name: bigquery-analytics
description: Query BigQuery data warehouse for analy
---

# BigQuery Analytics

## Available Datasets

- **Finance**: Revenue, billing, subscriptions → See
- **Product**: Feature usage, API calls, errors → Se
- **Sales**: Opportunities, pipeline, accounts → See
- **Marketing**: Campaigns, attribution, email → See

## Common Patterns

### Always Filter Test Data

```sql
WHERE account_id NOT IN (
  SELECT account_id FROM `company.internal.test_acco
)
```

### Date Handling

Use `DATE(timestamp_field)` for daily aggregations.

## Finding Specific Tables

```bash
````
````

```
grep -i "revenue" reference/finance.md
grep -i "signup" reference/product.md
```

The folder structure:

```
bigquery-analytics/
├── SKILL.md
└── reference/
    ├── finance.md
    ├── product.md
    ├── sales.md
    └── marketing.md
```

When someone asks about revenue, the AI reads `SKILL.md`, spots the reference to `finance.md`, and loads just that file. The other three stay on disk, costing zero tokens.

Example 5: Git Release Workflow

```
---
name: git-release
description: Create consistent releases and changelo
license: MIT
compatibility: opencode
metadata:
  audience: maintainers
```

```
  workflow: github
---

# Git Release Process

## What I Do

- Draft release notes from merged PRs
- Propose a semantic version bump
- Provide copy-pasteable `gh release create` comman

## When to Use Me

Use this when preparing a tagged release. I'll ask

## Workflow

1. **Identify changes since last release**
   ```bash
   git log $(git describe --tags --abbrev=0)..HEAD -
```

2. **Categorise changes**
   - Breaking changes → major bump
   - New features → minor bump
   - Bug fixes → patch bump

3. **Generate release notes**
   Group by category: Features, Fixes, Documentatio

4. **Create release**
```

````
   ```bash
   gh release create v1.2.3 --title "v1.2.3" --notes
   ```

## Release Notes Template

```markdown
## What's Changed

### Features
- feat: description (#PR)

### Fixes
- fix: description (#PR)

### Documentation
- docs: description (#PR)

**Full Changelog**: https://github.com/org/repo/comp
```
````

Example 6: Permission-Restricted Skill

Some skills shouldn't be available to all agents. You can restrict access in `opencode.json`

```
{
  "permission": {
    "skill": {
```

```
      "*": "allow",
      "internal-*": "deny",
      "experimental-*": "ask"
    }
  }
}
```

Wildcards work as you'd expect: `internal-*` matches `internal-docs`, `internal-tools`, `internal-secrets`.

```
---
name: internal-deployment
description: Deploy to production infrastructure. Us
---

# Production Deployment

⚠️ This skill has access to production systems.

## Deployment Commands

[Sensitive commands here...]
```

Example 7: Per-Agent Permission Overrides

Different agents can have different permissions. For a custom agent (in its frontmatter):

```
---
name: ops-agent
description: Infrastructure operations agent
permission:
  skill:
    "internal-*": "allow"
    "experimental-*": "deny"
---
```

For built-in agents (in `opencode.json`):

```
{
  "agent": {
    "plan": {
      "permission": {
        "skill": {
          "internal-*": "allow"
        }
      }
    }
  }
}
```

This lets your planning agent access internal documentation

while keeping other agents restricted.

Example 8: Complex Skill with Everything

Here's a more complete example showing multiple features together:

````
---
name: code-review
description: Review pull requests using team coding
license: MIT
compatibility: Works with TypeScript, Python, Go, ar
metadata:
  author: engineering-team
  version: "3.0.0"
  last-updated: "2025-11-15"
---

# Code Review Guidelines

## Review Checklist

Copy this checklist and check off items as you revie

```
Review Progress:
- [ ] Security: No hardcoded secrets, proper input v
- [ ] Performance: No N+1 queries, appropriate cachi
- [ ] Testing: Adequate coverage, edge cases handlec
- [ ] Documentation: Public APIs documented, complex
````
````

```
- [ ] Style: Follows team conventions (see reference
```

Security Review

For authentication/authorization changes, see [refer

Red flags to watch for:
- Direct SQL queries (prefer ORM)
- User input in shell commands
- Disabled security headers
- Commented-out authentication

Performance Review

For database-heavy changes, run:
```bash
python scripts/analyze-queries.py path/to/changed/f
```

Watch for:
- Unbounded queries (missing LIMIT)
- Missing indexes on filtered columns
- Sequential API calls that could be batched

Language-Specific Guidelines

- **TypeScript**: [reference/typescript.md](referenc
- **Python**: [reference/python.md](reference/pytho
- **Go**: reference/go.md
- **Rust**: reference/rust.md

```
## Feedback Style

Be direct but kind. Instead of:
> "This is wrong"

Say:
> "This could cause issues when X. Consider Y instea

Always explain the *why*.
```

Folder structure:

```
code-review/
├── SKILL.md
├── scripts/
│   └── analyze-queries.py
└── reference/
    ├── security-checklist.md
    ├── style.md
    ├── typescript.md
    ├── python.md
    ├── go.md
    └── rust.md
```

OpenCode vs the Anthropic Standard: What's Different?

OpenCode implements the Agent Skills standard with a few platform-specific additions:

The good news: if you write a skill that follows the standard format and put it in `.claude/skills/`, it'll work in both OpenCode and Claude Code. The OpenCode-specific features (permissions, per-agent overrides) are opt-in.

Disabling Skills Entirely

Sometimes you want an agent that can't use skills at all. For custom agents:

```
---
name: simple-chat
description: Basic chat without skill access
tools:
  skill: false
---
```

For built-in agents:

```
{
  "agent": {
    "plan": {
      "tools": {
        "skill": false
      }
    }
  }
}
```

When disabled, the `<available_skills>` section disappears from the system prompt entirely.

How the AI Discovers and Loads Skills

Knowing how the loading mechanism works helps you write better descriptions.

At startup, OpenCode generates something like this in the system prompt:

```
<available_skills>
  <skill>
    <name>conventional-commits</name>
    <description>Generate commit messages following
  </skill>
  <skill>
    <name>code-review</name>
    <description>Review pull requests using team coc
```

```
  </skill>
</available_skills>
```

When the AI decides a skill is relevant, it calls:

```
skill({ name: "conventional-commits" })
```

This loads the full `SKILL.md` into context. From there, the AI can read additional referenced files as needed.

This is why your description matters so much. A vague

description like “helps with code” won’t trigger reliably. A specific one like “Generate commit messages following conventional commit format” tells the AI exactly when to activate.

Mistakes I’ve Seen (And How to Avoid Them)

Mistake 1: Nested Reference Chains

Don’t do this:

```
SKILL.md → references advanced.md → references deta
```

The AI might partially read files when they’re multiple levels deep, using commands like `head -100` instead of reading the whole thing.

Keep references one level deep:

```
SKILL.md → references advanced.md
SKILL.md → references details.md
SKILL.md → references api-reference.md
```

Mistake 2: Over-Explaining

The AI is already smart. You don't need to explain what a PDF is or how Python imports work.

Too verbose:

```
PDF (Portable Document Format) files are a common f
contains text, images, and other content. To extrac
you'll need to use a library. There are many librar
```

Just right:

````
Use pdfplumber for text extraction:
```python
import pdfplumber
with pdfplumber.open("file.pdf") as pdf:
    text = pdf.pages[0].extract_text()
```
````

````
### Mistake 3: Vague Descriptions

**Bad:**
```yaml
description: Helps with documents
```
````

Good:

```
description: Extract text and tables from PDF files,
```

The description should include both what the skill does AND when to use it, with specific keywords the AI can match against.

Mistake 4: Windows-Style Paths

Always use forward slashes, even on Windows:

- ✅ `scripts/helper.py`
- ❌ `scripts\helper.py`

Unix-style paths work everywhere. Windows paths break on Mac and Linux.

Mistake 5: Magic Numbers in Scripts

If your skill includes scripts, document why configuration values exist:

Bad:

```
TIMEOUT = 47  # Why 47?
RETRIES = 5   # Why 5?
```

Good:

```
# HTTP requests typically complete within 30 seconds
# Longer timeout accounts for slow connections
REQUEST_TIMEOUT = 30

# Most intermittent failures resolve by the second r
# Three retries balances reliability vs speed
MAX_RETRIES = 3
```

Mistake 6: Too Many Options

Don't overwhelm with choices:

Bad:

```
You can use pypdf, or pdfplumber, or PyMuPDF, or pdf
```

Good:

````
Use pdfplumber for text extraction:
```python
import pdfplumber
````

For scanned PDFs requiring OCR, use `pdf2image` with
````

`pytesseract` instead.

Pick a default. Mention alternatives only when they

Testing Your Skills

Anthropic recommends an iterative approach using two

1. **Claude A** helps you write and refine the skil
2. **Claude B** (a fresh instance with the skill loa
3. You observe Claude B's behaviour and bring insigh

Here's the workflow:

<!-- Option A: Comprehensive iteration flow -->
```mermaid
flowchart LR
    A[Work through task<br/>without skill] --> B[Ide
    B --> C[Ask Claude A to<br/>create skill]
    C --> D[Review for<br/>conciseness]
    D --> E[Test with<br/>Claude B]
    E --> F{Working<br/>well?}
    F -->|No| G[Observe failures]
    G --> H[Refine with<br/>Claude A]
    H --> E
    F -->|Yes| I[Ship it]
```

Building Evaluations

For **critical** skills, create structured evaluations:

```
{
  "skills": ["pdf-processing"],
  "query": "Extract all text from this PDF and save
  "files": ["test-files/document.pdf"],
  "expected_behavior": [
    "Successfully reads the PDF using appropriate li
    "Extracts text from all pages without missing ar
    "Saves extracted text to output.txt in readable
  ]
}
```

Run the evaluation, compare against expected behaviour, and iterate.

Quick Reference: Frontmatter Fields

```
---
# Required
name: skill-name           # 1-64 chars, lowercase,
description: What it does and when to use it  # 1-10

# Optional
license: MIT               # Or "Proprietary. See LI
compatibility: Requires poppler-utils  # Environment
metadata:                  # Arbitrary key-value pa
  author: your-name
  version: "1.0"
---
```

Quick Reference: File Locations